BharatSahayak: Project Documentation

Team Binary Builders (PawanSingh Jagdish Bhandari, Rahul H. Bramhankar) · Catalyst Hack 2026

Live project: https://d21lokneek591y.cloudfront.net · **Source code:** https://github.com/PawanBhandari03/BharatSahayak-Catalyst

Contents

1. Project overview
2. Key features
3. Technology used
4. How it works
5. Backend API
6. Deployment on AWS
7. Run it on your computer
8. Project folders
9. Screenshots
10. Future scope
11. Team

1. Project overview

The problem. India has hundreds of government schemes for farmers, students, women, workers and senior citizens. Many people never find out which schemes they can get. The portals are hard to use, and the rules (age, income, work, state, category) are hard to check by hand.

Our solution. BharatSahayak is an AI-powered assistant that tells a person which government schemes they may qualify for. The person answers a few simple questions, on the **website** or on **WhatsApp**, in **English, Hindi or Marathi**. They get a list of matching schemes with the benefit amount, the documents to arrange, and an official link to apply.

Example. Savitribai, 52, is a widow and a farmer in Osmanabad. She answers five questions and sees every scheme she may qualify for, such as PM-KISAN and the widow pension.

2. Key features

Feature	What it does
Simple 3-step form	Only 5 answers are needed: age, gender, state, work and income. No Aadhaar number, no OTP, no password
Instant results	Matching runs in the browser, so results show in under a second
Benefit dashboard	Matched schemes, documents to arrange, a benefit wallet and a timeline of future benefits

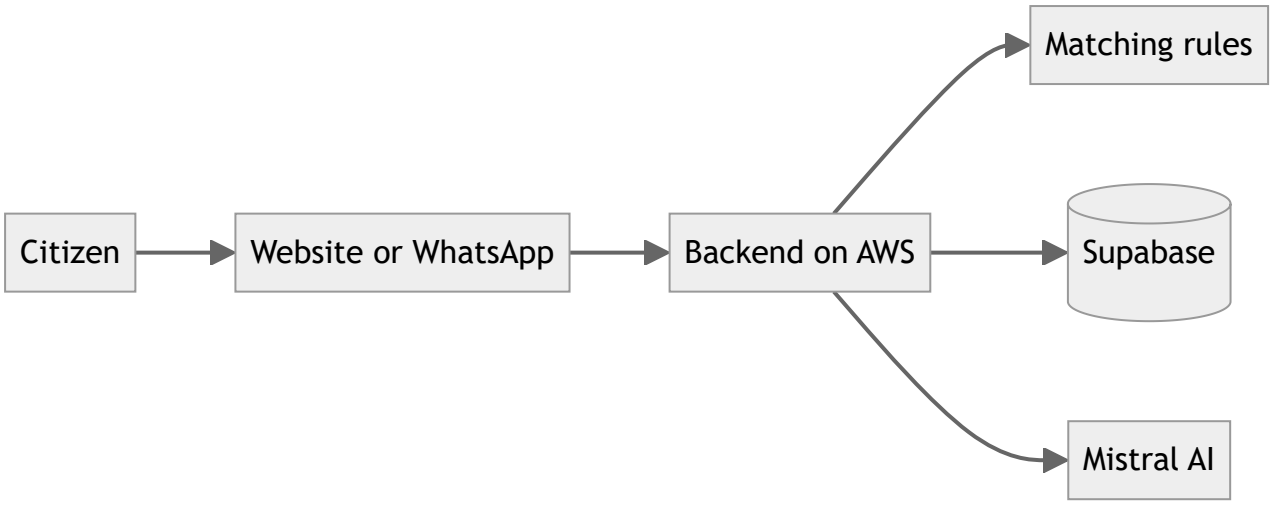
Feature	What it does
AI summary	A short, friendly summary of the top schemes, written by Mistral AI in the chosen language
WhatsApp bot	Chat with the bot in your language and receive your matching schemes in the chat
Scheme browser	Search and filter all 50 schemes by category, state, age, income and occupation
Three languages	The whole website works in English, Hindi and Marathi
Demo mode	A "Fill demo data" button on every form step, with three example people

3. Technology used

Part	Technology
Website	React 18, Vite, Tailwind CSS, Framer Motion, Recharts
Backend	Node.js, Express
AI	Mistral AI (model `open-mistral-nemo`)
WhatsApp	Twilio WhatsApp sandbox
Database	Supabase (PostgreSQL)
Hosting	AWS: Amazon EC2 server and Amazon CloudFront
Matching rules	One shared JavaScript file used by both the website and the backend
Tests	Node.js built-in test runner (31 tests)

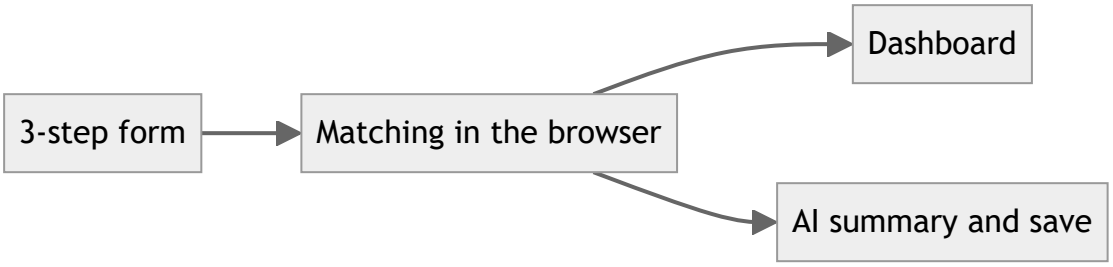
4. How it works

4.1 The whole system



The citizen uses the website or WhatsApp. Both talk to one backend running on AWS. The backend uses the matching rules to find schemes, saves profiles in Supabase, and asks Mistral AI to write summaries.

4.2 Website



1. The person fills the three-step form. The pages are: **About you**, **Work and income**, and **Your family** (optional).
2. The website loads the 50 schemes from the backend and the matching rules run **inside the browser**, so the answer appears at once.
3. The dashboard shows the matched schemes, the documents to arrange, the benefit wallet and the timeline.
4. In the background, the backend asks Mistral AI for a short summary and saves the profile in Supabase. The results never wait for these two steps.

Website pages: Home, Register (the form), Dashboard, Schemes, Journey and About.

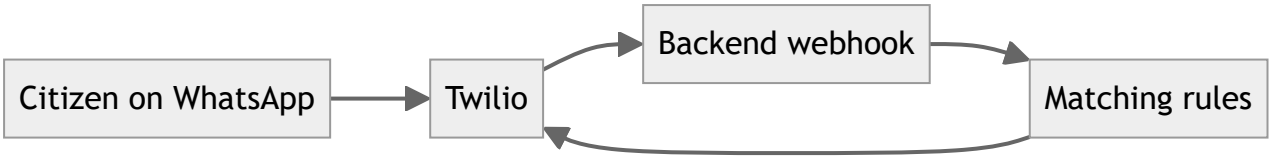
4.3 How the matching rules work

Every scheme is stored with simple rules: minimum and maximum age, gender, state, work, category, income limit, and whether it needs farm land or a ration card. The rules file (`shared/eligibility.mjs`) compares a person's answers with these rules and gives each scheme one of three answers:

- **Yes**: the person fits every rule.
- **Maybe**: the person fits the rules we know, but one answer is missing.
- **No**: the person clearly does not fit, and the reason is shown.

The **AI does not choose the schemes**. The rules choose them. The AI only writes the summary. This keeps the answers the same on the website and on WhatsApp, and stops the AI from inventing a scheme.

4.4 WhatsApp bot (Twilio)



Twilio is the service that connects WhatsApp to our backend. Here is what happens:

1. The citizen sends a message to the Twilio WhatsApp number.
2. Twilio forwards the message to our backend at `/api/whatsapp/webhook`.
3. The bot asks for a language (English, Hindi or Marathi), then age, gender, state, work, income and category, one question at a time. Mistral AI keeps the conversation friendly.
4. When all answers are collected, the matching rules pick the schemes.
5. The backend sends the reply through Twilio: up to five matching schemes, in one short message.

A daily limit on messages (default 80) stops the server before the free Twilio allowance runs out. The bot runs on Twilio's **sandbox**: each tester sends the join message once, and re-sends it after about three days.

4.5 The scheme data



There is no public official API for scheme eligibility, so we keep our own list of **50 schemes** (38 central, 11 Maharashtra and 1 for Jammu & Kashmir) in one file, `scheme-data/schemes.json`. Both the website and the WhatsApp bot read the same file.

Each scheme has:

Field	Meaning
`id`, `name`	A short id and the name in English, Hindi and Marathi
`category`, `level`, `state`	Topic, central or state scheme, and which state
`amount`, `frequency`, `benefitSummary`	The benefit in rupees, how often, and a short description in three languages
`eligibility`	The rules used for matching
`documents`, `howToApply`	Practical help for the applicant

Field	Meaning
`officialUrl`	The official website where the person can apply

Helper scripts in the project: `validate-schemes` checks the file format, `draft-scheme` asks Mistral AI to draft a new record from an official page, and `recheck-schemes` compares schemes with their official pages and writes a report.

4.6 Database (Supabase)

Supabase (PostgreSQL) stores two tables:

- **profiles**: the answers a person gave (age, gender, state, work, income and so on).
- **user_recommendations**: the matched schemes and the AI summary for that profile.

The table definitions are in `backend/db/schema.sql`. The backend connects to Supabase with a secret key that is kept only on the server. No Aadhaar number or OTP is ever asked for.

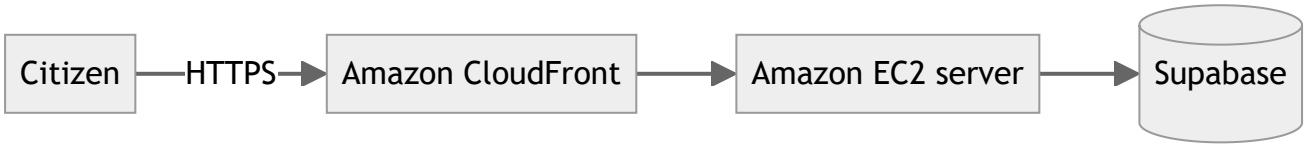
---

5. Backend API

Method	Path	What it does
GET	`/health`	Quick check that the server is running
GET	`/api/schemes`	All 50 schemes
GET	`/api/schemes/:id`	One scheme
POST	`/api/match-schemes`	Matches a profile with the schemes
POST	`/api/ai-recommend`	Writes the AI summary and saves the profile
POST	`/api/whatsapp/webhook`	Receives WhatsApp messages from Twilio

---

6. Deployment on AWS



The whole project runs on AWS:

Item	Value
Region	Asia Pacific (Sydney), `ap-southeast-2`

Item	Value
Server	**Amazon EC2** `t3.micro`, Amazon Linux 2023. Node.js runs the backend **and** serves the website. The `pm2` tool keeps it running
Secure link	**Amazon CloudFront** sits in front of the server and gives us the HTTPS address
Cost control	AWS Budgets alert, and free-tier services
Live link	https://d21lokneek591y.cloudfront.net

Steps we followed

1. Launched the EC2 instance and opened port 5000 in its security group.
2. Ran `deploy/setup-ec2.sh` once. It installs Node.js, git and pm2, and downloads the code.
3. Put the secret keys in `backend/.env` on the server (never in GitHub).
4. Ran `deploy/update.sh`. It pulls the latest code, builds the website and starts the server.
5. Created the CloudFront distribution to serve the site over HTTPS.

To release an update: push the code to GitHub, then run `deploy/update.sh` on the server.

---

7. Run it on your computer

You need Node.js 20.19 or newer, a Mistral AI key, and (for the database and WhatsApp) Supabase and Twilio accounts.

```
git clone https://github.com/PawanBhandari03/BharatSahayak-Catalyst.git
cd BharatSahayak-Catalyst

# Backend
cd backend
cp .env.example .env        # then fill in your keys
npm install
npm start                   # http://localhost:5000

# Website (open a second terminal)
cd frontend
cp .env.example .env        # VITE_API_URL=http://localhost:5000
npm install
npm run dev                 # http://localhost:5173
```

Settings in `backend/.env`

Name	Meaning
`PORT`	Server port (5000)

Name	Meaning
`MISTRAL_API_KEY`, `MISTRAL_MODEL`	Mistral AI key and model name
`SUPABASE_URL`, `SUPABASE_SERVICE_KEY`	Database address and secret key
`TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `TWILIO_WHATSAPP_FROM`	Twilio keys and the WhatsApp number
`WHATSAPP_DAILY_CAP`	Most WhatsApp messages sent per day

To create the database tables, open the Supabase SQL Editor and run `backend/db/schema.sql` once. To run the tests: `cd backend && npm test`.

Setting up WhatsApp with Twilio

1. Put the Twilio keys in `backend/.env`.
2. In the Twilio Console, go to Messaging, Try it out, Send a WhatsApp message, **Sandbox settings**.
3. Set **When a message comes in** to `https://<your-server>/api/whatsapp/webhook` (our live one is `https://d21lokneek591y.cloudfront.net/api/whatsapp/webhook`), method **POST**, and save.
4. From WhatsApp, send **`join spent-opinion`** to **+1 415 523 8886** (or scan the QR code on the website's home page).
5. Send **Hi**. The bot starts the conversation.

---

8. Project folders

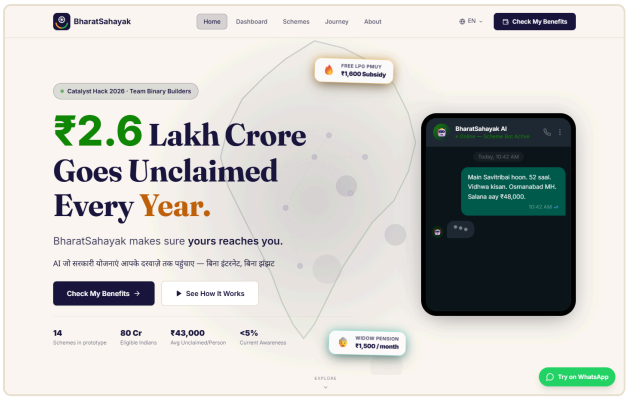
```
BharatSahayak-Catalyst/
├── frontend/        React website (pages, components, Hindi and Marathi text)
├── backend/         Node.js server (routes, controllers, services, tests, db/schema.sql)
├── shared/          eligibility.mjs, the matching rules used by website and backend
├── scheme-data/     schemes.json (the 50 schemes) and its format
├── scripts/         validate, draft and re-check scripts for scheme data
├── deploy/          setup-ec2.sh and update.sh for AWS
└── docs/            this documentation and the screenshots
```

---

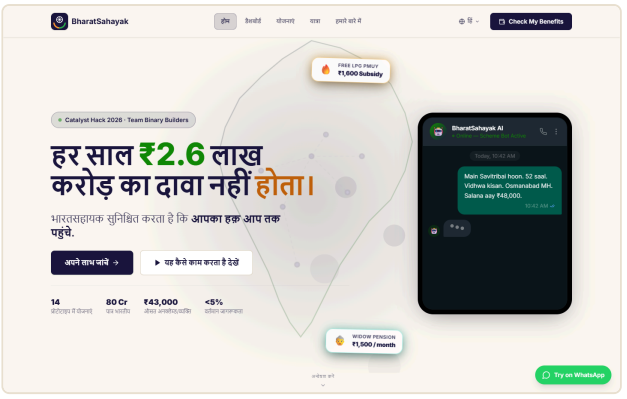
9. Screenshots

The demo data is the "Savitribai" example person.

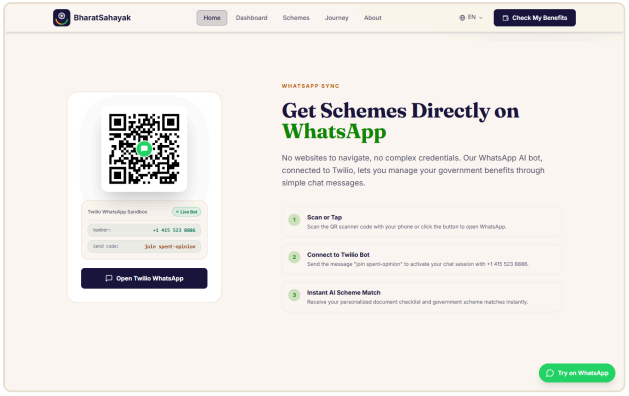
Home page



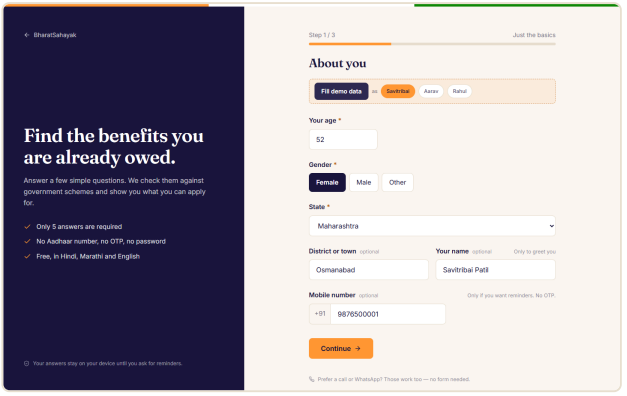
Home page in Hindi



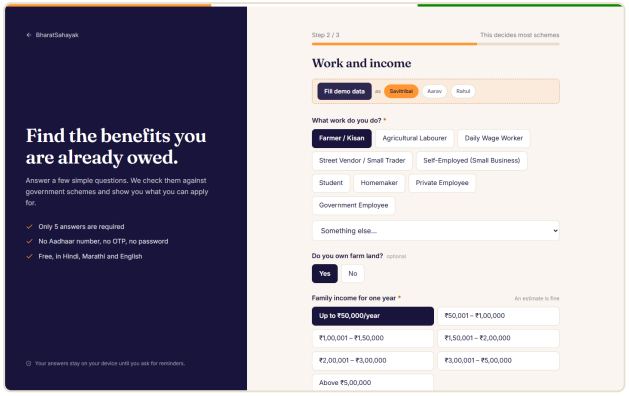
WhatsApp bot: scan the QR code



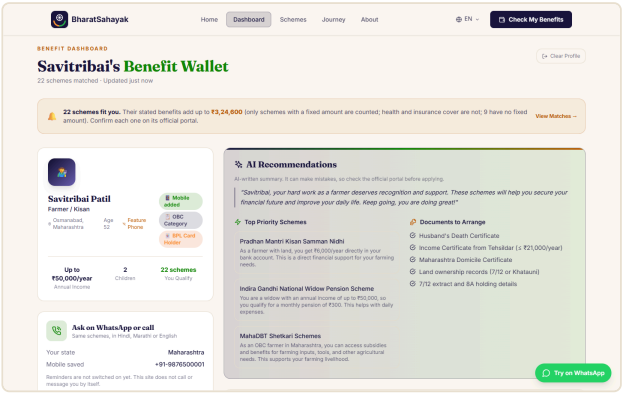
Form, step 1 (Fill demo data)



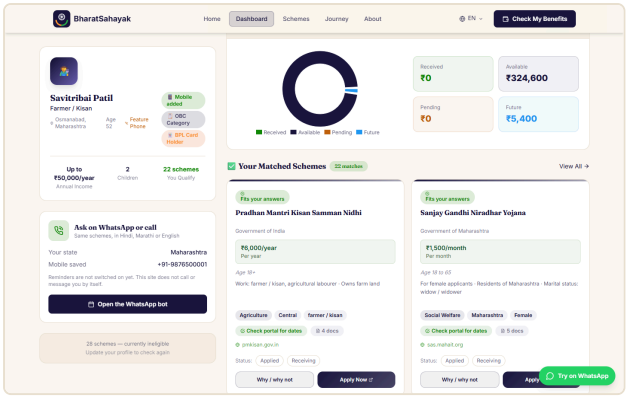
Form, step 2



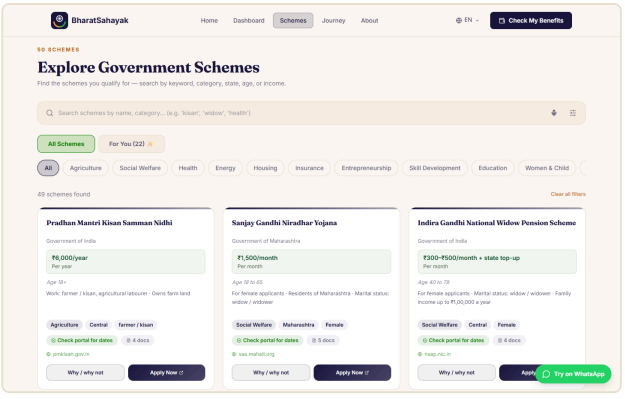
Benefit dashboard with the AI summary



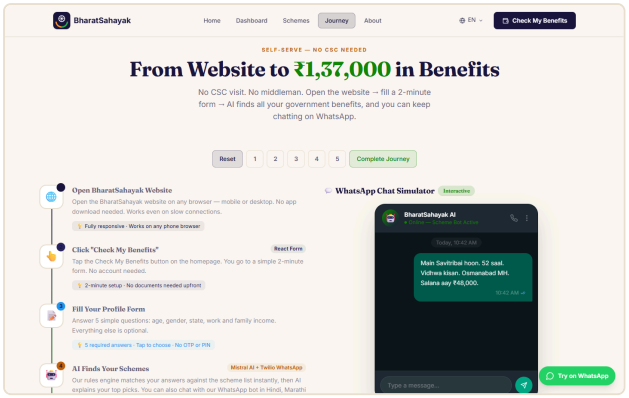
Dashboard: matched schemes



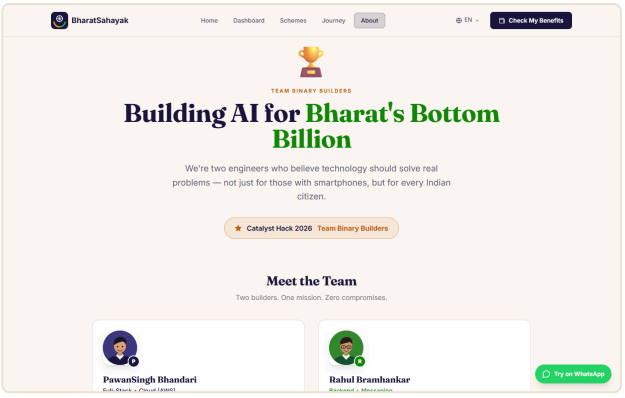
All schemes, with filters



Journey page



About the team



10. Future scope

- **Phone calls (Twilio Voice).** Citizens with basic keypad phones could call a number, speak in their language and hear their schemes. The groundwork exists, and it needs speech recognition and a paid Twilio phone number. This is not built because the deployment focus was on the AWS-hosted website and WhatsApp bot.
- **Reminders.** WhatsApp messages before a scheme's deadline, and alerts for life events such as a birth or retirement.
- **More schemes and states**, and more languages and regional dialects.
- **Scheme descriptions in Hindi and Marathi.** The names and benefit lines are already in three languages.
- **Help through Common Service Centres** for people who need assistance to apply.

11. Team

Team Binary Builders, Catalyst Hack 2026 (Generative AI).

Member	Contribution
PawanSingh Jagdish Bhandari	React website, Node.js backend, Supabase database, AWS deployment
Rahul H. Bramhankar	Twilio WhatsApp integration, scheme data pipeline, backend and frontend support